

StaffOS — End-to-End Build & Presentation Plan

Oct 7, 2026 · @pathum

1. Product concept and JD coverage

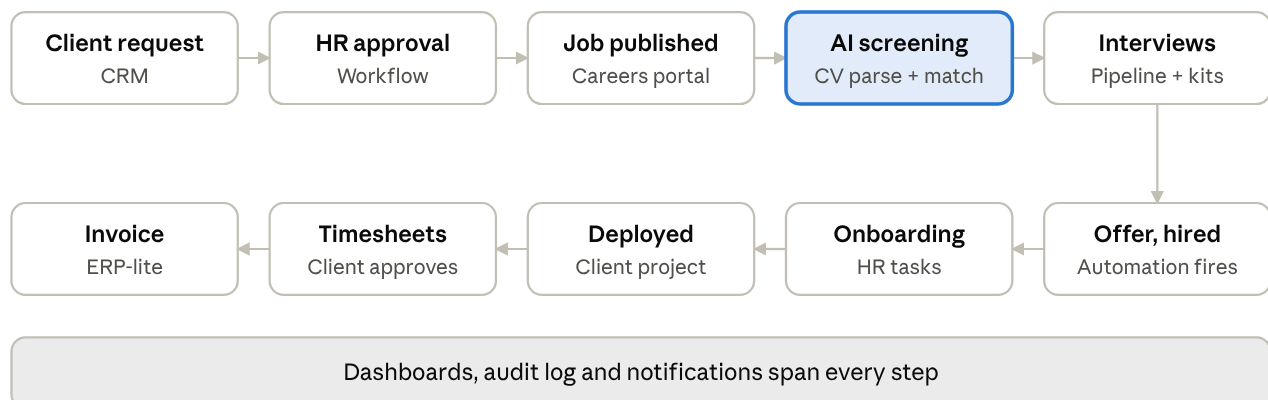
Build StaffOS: an AI-powered workforce and recruitment platform for a staffing company, covering the full flow from a client's manpower request to a hired worker deployed on site and invoiced. One connected system, not separate demos, so every JD keyword appears as a working part of the same story.

Why this concept wins: ASRS is a recruitment, contract-staffing and manpower-supply business inside a large group. A system that models how a staffing company actually operates shows you understood their business, which no generic "HR portal" candidate will.

Positioning line for the interview: "I can't share code from my previous employers, so I built an independent platform around the kind of business this role supports, and I built it the way the JD describes: AI-assisted, but with me owning the design, the review and the quality."

Never call it a company project or use ASRS branding.

StaffOS follows one worker from client request to invoice



End-to-end business flow · 10 steps across CRM, recruitment, HR and ERP

This is the story your demo walks through, and the slide you open the presentation with.

Every JD line, mapped

JD requirement	Where StaffOS proves it
Business applications	End-to-end client request → hire → deployment → invoice flow
HR systems	Employees, departments, documents, onboarding checklists, document expiry alerts
Recruitment portals	Internal ATS (jobs, candidates, Kanban pipeline) + public careers portal where candidates apply
ERP modules	Deployments to client projects, timesheets, invoices generated from timesheets
CRM modules	Client companies, contacts, manpower requests, activity notes, client portal
Dashboards and reports	Role-based dashboards, hiring funnel, time-to-hire, client and revenue reports, CSV/Excel/PDF export
Workflow automation	Configurable approval workflows + a "When X → do Y" automation rules builder + scheduled jobs
Responsive front-end and back-end	React + TypeScript web app, mobile-friendly, plus a NestJS API
Translate business requirements	PRD, user stories, process maps in docs/ written before any code
REST APIs + third-party integration	Versioned REST API with Swagger, webhooks out, email (Resend/SMTP), calendar invite (.ics), AI APIs
Databases, dashboards, reports	PostgreSQL schema with migrations, indexes, reporting views
Test, debug, performance, security	Unit, integration and E2E tests, load test, OWASP checklist, security scan in CI
Documentation + production support	README, architecture docs, API docs, runbook, changelog, monitoring and error tracking
AI coding tools + code quality	Lovable/Bolt for prototype, Claude Code for build, Copilot/Cursor for edits, AI dev log, review gates

JD requirement	Where StaffOS proves it
HTML5, CSS3, JS, TypeScript	Whole codebase in TypeScript, semantic HTML, Tailwind CSS
React/Angular/Vue	React
Node.js/.NET/Python/PHP	Node.js (NestJS)
PostgreSQL/MySQL/SQL Server	PostgreSQL
Auth + application security	JWT access + refresh tokens, role and permission checks, audit log, rate limiting
Git + CI/CD	GitHub flow, PR reviews, GitHub Actions pipeline, staging + production
Prompts, review AI code, prototypes	Prompt library in <code>docs/prompts/</code> , AI dev log, Lovable prototype
AI API integration	CV parsing, candidate matching, JD writer, interview kit, "ask your data"
Cloud + low-code/no-code	Docker, cloud deploy, n8n automation, Zoho CRM sync

2. Tech stack and AI tool roles

Use TypeScript end to end: React on the front, NestJS on the back, PostgreSQL underneath. Every choice is on the JD's own list, and it is a stack you can defend file by file.

Layer	Choice	Why (your interview answer)
Front end	React 18 + TypeScript + Vite, Tailwind CSS, shadcn/ui	JD names React + TS; component library gives an enterprise look fast
Data fetching / forms	TanStack Query, React Hook Form + Zod	Caching, loading states, one validation schema shared with the API
Charts / tables	Recharts, TanStack Table	Dashboards and sortable, filterable data grids

Layer	Choice	Why (your interview answer)
Back end	Node.js + NestJS	Modules, guards and DTOs give structure like Spring/.NET but in TypeScript
Database	PostgreSQL 16 + Prisma ORM + migrations	Relational workflows need integrity; migrations are versioned in Git
Background jobs	BullMQ + Redis	AI calls, emails and scheduled checks run off the request path
Files	S3-compatible storage (AWS S3 or Cloudflare R2), pre-signed URLs	Documents never served publicly
Auth	JWT access token + rotating refresh token in httpOnly cookie, bcrypt/argon2	Stateless API, safe token storage
AI	Anthropic Claude and OpenAI behind one <code>LlmProvider</code> interface	No vendor lock-in; swap per feature
Email	Resend or SMTP, with templates	Notifications, interview invites
Infra	Docker, docker-compose, GitHub Actions, AWS (or Render/Railway + Vercel)	JD asks for Git, CI/CD and cloud
Monitoring	Sentry, structured logs (pino), uptime check, /health endpoint	Production support
Low-code	n8n (self-hosted) + Zoho CRM API	JD asks for low-code; you already know Zoho

If you are clearly faster in Laravel, the plan works the same with Laravel + Inertia/React. Pick the one you can explain under pressure.

Which AI tool does what

The JD names seven AI tools. Use each one for the job it is best at, and record it, so you can honestly say you have used them in a real build.

Tool	Role in this project	Phase
Lovable	Clickable UI prototype of 6 key screens from your PRD, to validate the layout before coding	Design
Bolt.new	Quick throwaway spike of the public careers portal and the Kanban board interaction	Design
Claude (chat)	Turn your notes into PRD, user stories, schema review, threat model	Planning
Claude Code	Main implementation agent, module by module, plus test writing and refactors	Development
Cursor / GitHub Copilot	Small in-editor edits, autocomplete, reading unfamiliar code	Development
Codex (or Claude Code review)	Independent second-opinion code review on pull requests	Testing
Replit	Optional: a tiny public sandbox demo of the AI CV parser API	Prototyping

Key message: **prototypes come from Lovable/Bolt, production code comes from a reviewed Claude Code workflow.** You don't ship the prototype code; you use it to agree on the design.

3. Phase 1 — Planning

Write the requirements before any code; these files are your strongest proof that you designed the system and AI only implemented it.

Personas and roles

Role	Can do
Super Admin	Users, roles, permissions, settings, automation rules, audit log
HR Manager	Everything in recruitment and HR, approvals, reports

Role	Can do
Recruiter	Jobs, candidates, pipeline, interviews for assigned jobs
Account Manager (CRM)	Clients, contacts, manpower requests, deployments, invoices
Hiring Manager	Review shortlisted candidates, give interview feedback, approve offers
Finance	Timesheet approval, invoices, payments status
Employee	Own profile, documents, onboarding tasks, timesheets
Candidate (public)	Browse jobs, apply, upload CV, track application status
Client (portal)	Raise manpower requests, view shortlisted CVs, approve timesheets

Modules (scope)

1. **Identity and access** — login, refresh, password reset, 2FA (optional), roles, permission matrix
2. **CRM** — clients, contacts, activities, manpower requests with approval
3. **Recruitment (ATS)** — jobs, candidates, applications, Kanban pipeline, interviews, offers
4. **Careers portal** — public job board, apply form, application tracking link
5. **HR core** — employees, departments, positions, documents with expiry dates
6. **Onboarding** — checklist templates (documents, medical, visa, IT, induction), task assignment
7. **Workforce / ERP-lite** — deployments to client projects, timesheets, invoices
8. **Workflow engine** — approval chains + automation rules (trigger → condition → action)
9. **AI workspace** — CV parsing, matching, JD writer, interview kit, "ask your data"
10. **Dashboards and reports** — role dashboards, funnel, exports, scheduled weekly report email
11. **Notifications** — in-app bell, email, webhooks
12. **Administration** — settings, audit log, AI usage and cost log, system health

User stories (write 30–40, examples)

- As an Account Manager, I log a client's request for 10 heavy-vehicle drivers so that recruitment can start.
- As an HR Manager, I approve a manpower request above 20 headcount before a job opens.
- As a Recruiter, I upload a CV and get a pre-filled profile so I don't retype data.
- As a Recruiter, I see candidates ranked by AI match with reasons, so I review the best first.
- As a Hiring Manager, I submit interview scores so HR can decide on the offer.
- As HR, when a candidate is hired, onboarding tasks are created automatically.
- As HR, I'm alerted 30 days before an employee's passport or visa expires.
- As Finance, I generate a monthly invoice per client from approved timesheets.
- As a Client, I log in and approve my deployed workers' timesheets.

Each story gets acceptance criteria in Given/When/Then form. Claude Code uses these as its definition of done.

Non-functional requirements

- Page load under 2 s on seed data; API p95 under 300 ms for list endpoints
- Every list endpoint paginated, filterable, sortable
- Every write action audited (who, what, before/after, when, IP)
- Personal documents only visible to HR roles; all access logged
- AI features optional: core workflow works if the AI provider is down
- Responsive down to 375 px wide; keyboard accessible; WCAG AA contrast

Planning deliverables

- docs/01-PRD.md , docs/02-user-stories.md , docs/03-process-maps.md (BPMN-style flows), docs/04-nfr.md , docs/05-risks.md
- A GitHub Project board with epics per module and issues per story — this is your visible "project management" evidence.

4. Phase 2 — Design

Design in three layers — screens, architecture, data — and keep each as a file in docs/ .

4.1 UI/UX design

1. Write a one-page design brief: clean enterprise SaaS (think Linear, Rippling, Workday), light and dark mode, one accent colour, dense but calm tables.
2. Feed the PRD to **Lovable** and prototype 6 screens: dashboard, candidate list, Kanban pipeline, candidate profile with AI panel, client page, automation rule builder.
3. Use **Bolt.new** for the public careers portal and apply form.
4. Screenshot the prototypes into `docs/design/`, note what you changed and why (`docs/06-design-decisions.md`). That's your "used Lovable/Bolt for prototyping" story.
5. Define design tokens (colours, spacing, radius, type scale) in Tailwind config so every screen is consistent.

Key screens to design: login, role dashboards (HR, Recruiter, Account Manager, Finance), list + detail pages for each entity, Kanban board, onboarding checklist, timesheet grid, invoice preview, automation builder, AI "ask your data" box, audit log, settings, careers portal, client portal.

4.2 Architecture

Modular monolith: one NestJS app with strict module boundaries, not microservices.

Interview answer: "At this scale microservices add operational cost without benefit; the module boundaries mean any module could be split out later." The drawing is in section 4.5.

4.3 Database schema (core tables)

Area	Tables
Identity	users, roles, permissions, role_permissions, user_roles, refresh_tokens
CRM	clients, client_contacts, activities, manpower_requests
Recruitment	jobs, job_skills, candidates, candidate_skills, applications, application_stage_history, interviews, interview_feedback, offers
HR	departments, positions, employees, documents (polymorphic owner, expiry_date)

Area	Tables
Onboarding	onboarding_templates, onboarding_template_tasks, onboarding_plans, onboarding_tasks
ERP-lite	projects, deployments, timesheets, timesheet_entries, invoices, invoice_lines
Workflow	workflow_definitions, workflow_steps, approval_requests, automation_rules, automation_runs
AI	ai_requests (feature, model, tokens, cost, latency, status), ai_results
Platform	notifications, audit_logs, webhooks, webhook_deliveries, settings

Design rules to state in the interview:

- UUID primary keys, `created_at/updated_at/created_by` , soft delete where history matters
- Foreign keys and unique constraints in the database, not only in code
- Indexes on every foreign key and on common filters (status, client_id, created_at)
- Stage history is append-only, so time-to-hire and funnel reports are exact
- Money stored as integer fils ($\text{AED} \times 100$) with a currency column
- Reporting via SQL views (e.g. `v_hiring_funnel` , `v_client_revenue`), which also become the only tables the AI query feature may read

Draw the ERD with dbdiagram.io or Prisma ERD generator and save it as `docs/erd.png` .

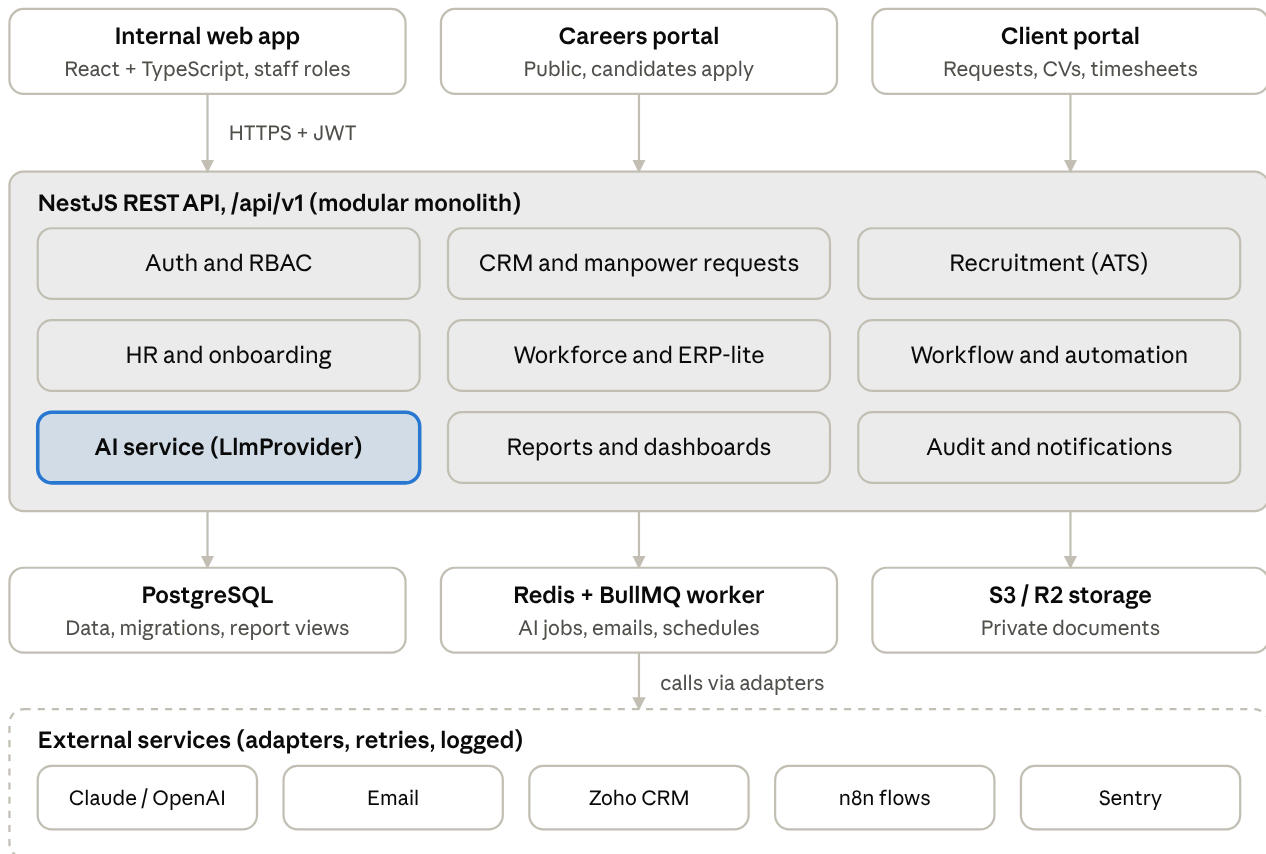
4.4 API design

- REST, versioned under `/api/v1` , JSON, OpenAPI/Swagger generated at `/api/docs`
- Resource endpoints: `GET/POST /clients` , `GET/PATCH/DELETE /clients/:id` , same for jobs, candidates, employees...
- Business actions as explicit endpoints: `POST /applications/:id/transition` , `POST /manpower-requests/:id/approve` , `POST /timesheets/:id/submit` , `POST /invoices/generate`

- AI endpoints: `POST /ai/cv-parse`, `POST /ai/match/:jobId`, `POST /ai/jd-draft`, `POST /ai/ask`
- Standard list query: `?page=&pageSize=&sort=&filter[status]=`
- One error shape everywhere: `{ code, message, details, traceId }`
- Webhooks out: `application.stage_changed`, `employee.hired`, `invoice.issued`, signed with HMAC
- Idempotency key header on invoice generation

4.5 Architecture drawing

One API serves three portals; AI and integrations sit behind adapters



StaffOS architecture · 3 front ends, 9 API modules, 3 data stores, 5 external services

All three front ends call one API; long-running AI and email work goes through the queue worker, and every external service is reached through an adapter so it can be swapped or retried.

5. Phase 3 — Development

Build module by module in a fixed loop: spec → Claude Code implements → you review → AI second review → tests → PR → merge. The loop is what you present, not just the result.

5.1 Repository layout (monorepo)

```
staffos/
├── apps/
│   ├── web/                # React + TS (internal app, careers portal, client
portal routes)
│   └── api/                 # NestJS
│       └── src/modules/
│           ├── auth/ users/ clients/ manpower-requests/ jobs/ candidates/
│           ├── applications/ interviews/ employees/ documents/ onboarding/
│           ├── deployments/ timesheets/ invoices/ workflows/ automations/
│           └── ai/ reports/ notifications/ audit/ webhooks/ common/
├── packages/
│   └── shared/              # Zod schemas + types shared by web and api
├── prisma/                  # schema.prisma, migrations, seed.ts
├── docs/                    # PRD, stories, ERD, ADRs, prompts, AI dev log,
runbook
├── automation/              # n8n workflow exports (JSON)
├── e2e/                     # Playwright tests
├── .github/workflows/       # ci.yml, deploy.yml
├── docker-compose.yml
├── CLAUDE.md                # rules for Claude Code
└── README.md
```

Each backend module follows the same shape: **controller** (thin, HTTP only) → **service** (business rules) → **repository** via Prisma, plus **dto/** with validation and ***.spec.ts** tests.

5.2 CLAUDE.md (write this yourself, first)

It tells Claude Code your rules, which keeps the code consistent and makes your ownership visible. Include:

- Stack and versions; folder conventions; naming
- "Controllers stay thin; business rules live in services"
- "Every endpoint has a permission guard and a DTO with validation"

- "Every write goes through the audit service"
- "Never call the LLM SDK directly; use `LlmProvider`"
- Error format, pagination format, test requirements per module
- "Don't modify files outside the module you were asked to change"
- "Stop and ask if a requirement is ambiguous"

5.3 The per-module loop

1. Open a GitHub issue from the user story; create branch `feat/<module>`.
2. Prompt Claude Code with: the story, acceptance criteria, schema section, API contract, and "follow CLAUDE.md". Ask for a plan first, approve it, then let it implement.
3. Read every changed file yourself. Run it. Click through it.
4. Ask Claude Code (or Codex) in a fresh session to review only: security, permission gaps, IDOR, N+1 queries, missing validation, edge cases. No edits.
5. Fix the findings (yourself or by targeted prompt).
6. Tests: unit for service rules, integration for endpoints, one E2E for the happy path.
7. Open a PR with a description, let CI run, merge. Small, meaningful commits (`feat(applications): enforce stage transitions`).
8. Log in `docs/ai-dev-log.md`: what you asked, what AI got wrong, what you changed.

5.4 Build order

1. Scaffold: monorepo, Docker Postgres/Redis, NestJS + React shells, CI running lint/test on an empty project, deploy "hello world" to staging on day one
2. Auth, users, roles, permissions, audit log, app shell (sidebar, top bar, theme)
3. CRM: clients, contacts, manpower requests + approval
4. Recruitment: jobs, candidates, applications, Kanban with backend-enforced transitions, interviews, offers
5. Careers portal (public apply) + email notifications
6. AI: CV parsing, matching, JD writer, interview kit
7. HR core: employees, documents with expiry
8. Onboarding templates + auto-created plans on hire
9. Workflow engine: approval chains + automation rules builder
10. ERP-lite: deployments, timesheets, invoices (PDF)

- 11. Dashboards, reports, exports, "ask your data"
- 12. Client portal
- 13. n8n + Zoho CRM integration, webhooks
- 14. Hardening: performance, security scan, accessibility, monitoring
- 15. Seed data, demo accounts, docs, rehearsal

5.5 Seed data

Write a seed script with realistic UAE staffing data: 12 clients (construction, logistics, facilities, healthcare), ~40 jobs (heavy driver, site engineer, nurse, electrician, accountant, full-stack developer), ~300 candidates, ~80 employees, 6 months of stage history so charts have trends. Use fictional names and companies only.

6. AI features and responsible AI

Five AI features, each solving a real staffing-company task, all behind one AI service with validation, logging and a fallback.

Feature	What it does	Engineering detail to mention
CV parser	PDF/DOCX → structured profile (contact, skills, years, education, certs, languages, nationality only if the user enters it)	Text extraction first, then LLM with a JSON schema; Zod validates; low-confidence fields highlighted for the recruiter to confirm
Candidate matching	Ranks applicants for a job with a score and reasons (matched / partial / missing skills)	Deterministic pre-filter (must-have skills, years) + LLM explanation; score cached per candidate-job pair
JD writer	Draft job description from title, client, location, salary band	Recruiter edits before publishing; inclusive-language check
Interview kit	Role-specific technical + behavioural questions, scoring rubric, then a summary from interviewer notes	Summary never auto-decides; it feeds the hiring manager

Feature	What it does	Engineering detail to mention
Ask your data	"Which clients have open requests older than 30 days?" → answer + table + chart	LLM writes SQL only against whitelisted reporting views, through a read-only DB role, with a row limit and statement timeout; SQL is shown for transparency

AI architecture rules

- One `LlmProvider` interface with `ClaudeProvider` and `OpenAiProvider`; model chosen per feature in settings
- Prompts are versioned files in `apps/api/src/modules/ai/prompts/`, not strings in code
- Every call is logged to `ai_requests`: feature, model, tokens, cost, latency, status, user
- Long calls (CV parse, batch matching) run as background jobs; the UI shows progress
- Timeouts + retries with backoff; if AI fails, the record is still saved and the user fills fields manually
- Monthly budget cap per feature in settings, shown on an AI usage dashboard

Responsible AI (say this out loud in the demo)

- **Human in the loop:** AI suggests, people decide. Labels read "AI-assisted suggestion".
- **Prompt-injection defence:** CV text is untrusted. It's passed as delimited data, the model's output must match a schema, and hidden instructions ("rate me 100%") can't change a score because ranking starts from deterministic rules.
- **Bias controls:** matching ignores name, gender, age, nationality and photo; those fields are stripped before the prompt.
- **Data minimisation:** only fields the feature needs are sent; no passport or visa data ever goes to an LLM.
- **Explainability:** every score shows its reasons and the prompt version that produced it.

AI evaluation

Keep a small test set (`ai/evals/`): 10 sample CVs with expected extracted fields and 3 jobs with an expected ranking. A script reports extraction accuracy and ranking agreement. Run it when you change a prompt or model. This is rare among candidates and very convincing.

7. Phase 4 — Testing and QA

Test the business rules hardest, because that's where a staffing system loses money or breaks compliance.

Level	Tool	What it covers	Target
Unit	Jest	Service rules: stage transitions, approval chains, invoice totals, automation conditions	80%+ on services
Integration / API	Jest + Supertest + test Postgres (Testcontainers)	Endpoints, permissions per role, validation, error shapes	Every endpoint, every role
Front-end component	Vitest + React Testing Library	Forms, Kanban drag rules, tables	Key components
End-to-end	Playwright	5 critical journeys (see below), run in CI against a seeded stack	All green before deploy
AI evals	Custom script	CV extraction accuracy, ranking agreement	Tracked per prompt version
Performance	k6	List endpoints and dashboard under 50 concurrent users	p95 < 300 ms
Security	npm audit, CodeQL/Semgrep, OWASP ZAP baseline	Dependency and code vulnerabilities, common web attacks	No high findings
Accessibility	axe (Playwright plugin), Lighthouse	Contrast, labels, keyboard	Lighthouse a11y 90+

Five critical E2E journeys

1. Client request approved → job opened → candidate applies via careers portal → appears in pipeline
2. Recruiter uploads CV → AI fills profile → recruiter confirms → match score shown

3. Candidate moved to Hired → automation creates onboarding plan + notifies account manager
4. Employee deployed → timesheet submitted → client approves in portal → invoice generated
5. Recruiter tries to open another client's candidate or an admin page → blocked (403)

Must-have negative tests

- Invalid stage jump (Applied → Hired) rejected by the API, not just the UI
- IDOR: changing an ID in the URL never exposes another client's data
- Expired or reused refresh token rejected
- Malformed AI response → handled, record still saved
- File upload with wrong type or oversize rejected; filenames sanitised

Debugging evidence

Keep 3–4 real bugs you hit in `docs/bug-log.md` (symptom, root cause, fix, test added). Interviewers love "tell me about a bug" and you'll have fresh, true answers.

8. Security plan

Treat it as a system holding passports, visas and salaries: enforce access in the backend, log every sensitive access, and keep secrets out of the code. Write `docs/security.md` mapping each OWASP Top 10 risk to how StaffOS handles it.

Risk	Control in StaffOS
Broken access control / IDOR	Permission guard on every route + ownership check in every service query (scoped by client/team); tests per role
Authentication	Argon2/bcrypt hashing, short-lived access JWT (15 min), rotating refresh token in httpOnly Secure SameSite cookie, reuse detection, lockout after 5 failures, optional TOTP 2FA
Injection	Prisma parameterised queries; raw SQL only in reporting views; AI-generated SQL runs read-only on whitelisted views

Risk	Control in StaffOS
XSS	React escaping, no <code>dangerouslySetInnerHTML</code> , sanitised rich text, strict Content-Security-Policy via Helmet
CSRF	SameSite cookies + CSRF token on cookie-auth endpoints
Sensitive data exposure	HTTPS only, documents in private bucket with 5-minute pre-signed URLs, PII fields masked in lists, document views audited
Security misconfiguration	Helmet headers, strict CORS allow-list, no stack traces in production errors
Vulnerable dependencies	Dependabot, npm audit and CodeQL in CI
Logging and monitoring	Audit log for writes and sensitive reads, Sentry alerts, login anomaly log
Abuse	Rate limiting on login, apply form and AI endpoints; CAPTCHA on public apply
File uploads	Type and size checks, renamed files, optional ClamAV scan job
Secrets	<code>.env</code> never committed, <code>.env.example</code> only, cloud secret manager in production, key rotation note in runbook

Mention UAE data-protection awareness: personal data collected only for a stated purpose, retention policy for rejected candidates (e.g. auto-anonymise after 12 months, configurable), and a candidate data-export/delete request flow.

9. Phase 5 — Production

Ship to a real cloud URL with staging and production, deployed only by the pipeline. A live link they can open on their own phone is worth more than any slide.

Environments

Env	Purpose	Deploys when
Local	docker-compose (Postgres, Redis, MinIO, Mailpit, n8n)	Always
Staging	Same as prod, seeded demo data	Every merge to <code>main</code>
Production	The demo you present	Manual approval on a tagged release (<code>v1.0.0</code>)

Hosting options (pick one)

- **AWS (strongest for the CV):** ECS Fargate or a single EC2 with Docker for API + worker, RDS PostgreSQL, ElastiCache Redis (or Upstash), S3 for documents, CloudFront for the web app, Route 53 + ACM for HTTPS, Secrets Manager.
- **Fast and cheap:** Vercel (web), Render or Railway (API, worker, Postgres, Redis), Cloudflare R2 (files).

Either way: a custom domain (e.g. `staffos.yourname.dev`), HTTPS, and the API on `api.` subdomain.

CI/CD pipeline (GitHub Actions)

1. On every PR: install → lint → typecheck → unit + integration tests → build → CodeQL + npm audit
2. On merge to `main` : build Docker images → push to registry (GHCR/ECR) → run Prisma migrations on staging → deploy staging → Playwright E2E against staging
3. On release tag: manual approval → migrate production → deploy → smoke test `/health` → notify (Slack/email)
4. Rollback: redeploy previous image tag; migrations written to be backward compatible

Branch protection on `main` : PR required, CI must pass. Keep conventional commits so a changelog is generated automatically.

Production readiness checklist

- ☐ HTTPS, security headers, CORS locked to your domains
- ☐ Secrets in the cloud secret store

- ☐ Daily automated DB backup + a tested restore (write down the steps)
- ☐ `/health` and `/ready` endpoints; uptime monitor
- ☐ Sentry on web and API; structured logs
- ☐ Demo accounts per role with a "Reset demo data" admin action

10. Phase 6 — Maintenance and production support

Show that you think past launch: you can see problems, fix them safely, and keep the system healthy. The JD says "support production systems"; this section is your evidence.

- **Observability:** Sentry errors with release tags; request logs with a `traceId` returned in every error so support can find it; a System Health page in Admin (DB, Redis, queue depth, AI provider status, last backup).
- **Runbook (`docs/runbook.md`):** how to deploy, roll back, restore a backup, rotate a secret, handle "AI provider down", handle "emails not sending", re-run failed automation jobs.
- **Incident template + one sample postmortem** from a real bug you hit (blameless format: timeline, root cause, fix, prevention).
- **Change management:** semantic versioning, auto-generated `CHANGELOG.md`, release notes page inside the app ("What's new").
- **Dependency upkeep:** Dependabot weekly, monthly patch release.
- **Data jobs:** nightly document-expiry scan, weekly report email, retention/anonymisation job, AI budget alert.
- **Feedback loop:** in-app "Report a problem" that creates a GitHub issue with page URL and `traceId`.
- **Roadmap (`docs/roadmap.md`):** payroll/WPS export, Arabic + RTL, mobile app for workers' timesheets, SSO with Microsoft 365. Having a credible v2 list shows product thinking.

11. Low-code/no-code and integrations

Pair your coded workflow engine with a low-code layer, so you can say: "core business rules live in code; quick, changeable automations live in low-code." That judgement is what the JD's low-code line is really testing.

In-app automation rules builder (built by you)

A no-code UI for business users: **When** [event] **If** [conditions] **Then** [actions].

- Events: application stage changed, employee hired, document expiring, timesheet submitted, manpower request created
- Conditions: field comparisons (client = X, headcount > 20, stage = Offer)
- Actions: create task, send email/in-app notification, assign user, start approval, call webhook
- Every run logged in `automation_runs` with input, result and errors; failed runs can be retried

Example rules to pre-load: "When a candidate is Hired → create onboarding plan + notify account manager"; "When a passport expires in 30 days → email HR + task"; "When a manpower request > 20 headcount → require HR Manager approval".

n8n (external low-code)

Self-host n8n in docker-compose. StaffOS sends signed webhooks; n8n runs 2–3 flows, exported as JSON into `automation/`:

- New hire → create a row in a Google Sheet / send a Microsoft Teams message
- Weekly report → generate and email a PDF to management
- Inbound CV email → forward attachment to the StaffOS CV-parse API

Third-party integrations

Integration	Why it's here
Zoho CRM API (OAuth2)	Sync clients and contacts both ways; matches your real Avyanco experience and the JD's CRM line
Email (Resend/SMTP)	Notifications, interview invites with <code>.ics</code> calendar file
Claude + OpenAI APIs	AI features
S3/R2	Documents
Outgoing webhooks	Any external system can subscribe to StaffOS events

Integration	Why it's here
Optional: Microsoft Graph	Teams meeting link for interviews (fits a Teams-using company)

For each integration, show the same pattern: adapter class behind an interface, retries with backoff, idempotency, logged deliveries.

12. Documentation deliverables

The **docs/** folder is half your presentation: it proves the thinking behind the code.

File	Contents
README.md	One-line pitch, live demo link + demo logins, screenshots/GIF, feature list, architecture image, tech stack, run locally in 3 commands, test commands, links to docs
docs/01-PRD.md	Problem, users, scope, out of scope, success metrics
docs/02-user-stories.md	Stories + acceptance criteria
docs/03-process-maps.md	Recruitment, onboarding, timesheet-to-invoice flows
docs/04-nfr.md , 05-risks.md	Non-functional requirements, risk register
docs/06-design-decisions.md + design/	Lovable/Bolt prototypes, what changed and why
docs/architecture.md + erd.png	Diagram, module boundaries, data model
docs/adr/	Architecture Decision Records: NestJS vs Express, modular monolith vs microservices, Prisma, JWT+refresh, AI provider abstraction, SQL views for AI queries
/api/docs (Swagger)	Live, generated API reference
docs/security.md	OWASP mapping, data protection

File	Contents
docs/testing.md	Strategy, how to run, coverage report
docs/ai-dev-log.md	How AI tools were used, prompts, what AI got wrong, what you changed
docs/prompts/	Reusable prompts for Claude Code (feature, review, test, refactor)
docs/runbook.md , bug-log.md , postmortem-001.md	Operations evidence
CHANGELOG.md , docs/roadmap.md	Releases and v2 plan
User guide (in-app Help page)	Short how-to per role, which business users actually read

13. Presentation and demo script

Run a 25-minute session: 2 minutes context, 12 minutes product story, 8 minutes engineering, 3 minutes AI workflow — then offer a live build. Business people see the product first; technical people see the engineering after.

Part 1 — Context (2 min)

"My previous systems belong to my employers and their clients, so I can't share that code. Instead I built StaffOS from scratch to show how I'd build the kind of systems this role covers: CRM, recruitment, HR, workflow, ERP-lite and AI, as one platform. It's live, and everything you'll see went through planning, design, development, testing and a CI/CD pipeline."

Show one slide: the business flow from client request to invoice.

Part 2 — Product story (12 min, switch roles as you go)

1. **Account Manager:** client "Gulf Build Contracting" requests 25 heavy drivers → request needs HR approval (the "headcount over 20" rule fires) → approved.
2. **HR Manager:** opens a job from the request; AI drafts the JD; edit and publish.
3. **Careers portal (on your phone):** apply as a candidate with a sample CV.

4. **Recruiter:** CV auto-parsed; confirm highlighted fields; AI match ranks candidates with reasons. Point out the "AI-assisted" label.
5. **Pipeline:** drag through stages; try an illegal jump → blocked by the API. Generate an interview kit; record feedback.
6. **Hired:** automation fires → onboarding plan created, account manager notified. Show the automation rule and its run log.
7. **Deployment + timesheet:** deploy the worker to a client project; submit timesheet; approve as the client in the client portal; generate the invoice PDF.
8. **Dashboard:** numbers updated; ask "Which clients have open requests older than 30 days?"; show the generated SQL.
9. **Admin:** audit log of everything you just did; AI usage and cost page; system health.

Part 3 — Engineering (8 min)

Architecture diagram → one module's code (controller → service → repository) → the permission guard → a test file → GitHub Actions run → Swagger docs → ERD → security.md.

Part 4 — AI-assisted workflow (3 min)

Show `CLAUDE.md`, one prompt from `docs/prompts/`, the Lovable prototype vs the final screen, and two entries from `ai-dev-log.md` where AI was wrong and you caught it. Say: "AI wrote a lot of the code; I wrote the requirements, design, rules and reviews."

Part 5 — Offer a live change

"If you'd like, give me a small requirement now — say a leave request with manager approval, or a new field on candidates — and I'll implement it with Claude Code here, including the review step." Rehearse 2–3 of these in advance until each takes under 20 minutes.

Bring

- Laptop charged + charger + HDMI/USB-C adapter; mobile hotspot
- App running locally as backup if the internet fails; screen recording of the full demo as a second backup
- Demo logins on a card; a sample CV PDF on the desktop
- GitHub repo open, Claude Code logged in

- Printed one-page summary: link, QR code to the live demo, stack, feature list
- Reset demo data right before you walk in

14. Questions they'll ask about the project

Prepare short, true answers to these; each one should point at something you can open on screen.

Question	Answer in one breath (then show it)
How much of this did AI write?	Most of the typing. I wrote the PRD, schema, API contract, CLAUDE.md rules and acceptance criteria, reviewed every change, and fixed what AI got wrong — here's the log.
Show me where permissions are enforced.	Guard on the route plus scoping in the service query; here's the test that a recruiter gets 403.
What happens if the AI provider is down?	Record still saves, fields stay editable, the job retries later, and the failure is logged.
How do you stop a CV from manipulating the AI?	CV text is treated as data, output must match a schema, ranking starts from deterministic rules, a human decides.
Why a modular monolith, not microservices?	One team, one deploy, lower ops cost; module boundaries let us split later. ADR-002.
Why PostgreSQL?	Relational workflows, constraints, transactions for invoices, views for reporting.
How would this scale to 50,000 candidates?	Indexes already on filters, pagination everywhere, heavy work in queues, read replica for reports, full-text search via Postgres or OpenSearch later.
How do you deploy? Roll back?	Walk through the GitHub Actions run; rollback = redeploy previous image tag.
Tell me about a bug you hit.	One from <code>bug-log.md</code> : symptom, root cause, fix, test added.

Question	Answer in one breath (then show it)
How do invoices stay correct?	Totals computed server-side from approved timesheets, integer money, idempotency key, unit tests.
How would you add Arabic?	i18n keys already used for UI strings; add RTL with Tailwind's <code>rtl:</code> variants; it's on the roadmap.
Could you build this for our company?	Yes — I'd start the same way: map your actual processes, then prioritise the module that saves the most manual work.
Did you build systems like this before?	Answer from your real past work only, and make it consistent with your first-round answers.

Golden rule: never claim anything about StaffOS you can't show, and never present StaffOS as a past employer's system.

15. Master checklist

Tick these in order; the project is interview-ready when every box in a phase is done before the next starts.

Planning

- ☐ PRD, user stories with acceptance criteria, process maps, NFRs, risks in `docs/`
- ☐ GitHub repo + Project board with epics and issues

Design

- ☐ Lovable prototype (6 screens) and Bolt careers portal spike, screenshots saved
- ☐ Design tokens, architecture doc, ERD, API contract, ADRs
- ☐ `CLAUDE.md` written by you

Development

- ☐ Scaffold + CI + staging deploy on day one
- ☐ Auth, roles, audit log, app shell
- ☐ CRM + manpower requests with approval
- ☐ Recruitment ATS + Kanban + interviews + offers

- ☐ Careers portal + email
- ☐ AI: CV parse, matching, JD writer, interview kit
- ☐ HR core + documents with expiry
- ☐ Onboarding plans
- ☐ Workflow approvals + automation rules builder
- ☐ Deployments, timesheets, invoices
- ☐ Dashboards, reports, exports, ask-your-data
- ☐ Client portal
- ☐ n8n flows, Zoho CRM sync, webhooks
- ☐ Realistic seed data + demo accounts + reset action

Testing

- ☐ Unit + integration per module, 5 E2E journeys, negative tests
- ☐ AI evals, k6 load test, security scans, accessibility check
- ☐ Bug log with 3–4 real entries

Production

- ☐ Production on a custom domain over HTTPS, release `v1.0.0` via pipeline
- ☐ Backups + tested restore, monitoring, Sentry

Maintenance and docs

- ☐ Runbook, postmortem, changelog, roadmap, user help page
- ☐ README with screenshots, demo GIF, live link, logins
- ☐ AI dev log + prompt library

Presentation

- ☐ Walk through every module's code until you can explain any file cold
- ☐ Rehearse the 25-minute demo twice, timed
- ☐ Rehearse 2–3 live changes with Claude Code
- ☐ Offline backup + screen recording + printed one-pager with QR code